AI-BASED ELECTRICITY DEMAND INTELLIGENCE SYSTEM FOR SRI LANKA

INDIVIDUAL THESIS

Individual component: Scenario Analysis and Decision Support for Electricity Demand Forecasting

Group ID: R26-IT010

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka



October 2026

Dewmini D.M.D.P

(IT22630216)

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Dewmini D.M.D.P

Student ID: IT22630216

Signature: ____  ____ Date: _____10/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	**Signature**	**Date**
Supervisor: Dr. Nathali Silva		
Co-supervisor (if any):		

ABSTRACT

Energy planners cannot wait for a heat wave, a cold spell or a price change to happen before deciding how to respond. They need to estimate in advance how electricity demand will react to unusual conditions. Most forecasting tools return a single predicted value and offer no way to test such what-if situations. This thesis presents the scenario analysis and decision-support component of the AI-Based Electricity Demand Intelligence System for Sri Lanka (Project R26-IT010). The component forecasts electricity demand, simulates what-if scenarios on top of the forecast, and turns each result into a risk level and a recommendation.

The study uses 189,888 electricity demand observations recorded every 15 minutes from January 2020 to May 2025, together with twelve weather, economic and time features. After mean imputation and training-only IQR outlier removal with a wide factor of 3.0, Linear Regression, Random Forest and Gradient Boosting were compared on identical data. A stacking ensemble, built from out-of-fold predictions with a Ridge meta learner tuned by Optuna over 50 trials, was added as a research extension. A scenario engine then simulates a heat wave, a cold spell, a price shock and an economic boom, and the component is delivered through a FastAPI backend and a React dashboard.

All models reached a test R^2 above 0.9999, with Linear Regression the most accurate. The tuned ensemble placed about 99.9% of its trust on the linear base learner, which confirms that demand in this dataset depends almost entirely on a near-linear relationship with temperature. Price and GDP scenarios produced almost no change in demand. A computed deployment rule recommended keeping Random Forest, but the ensemble was deployed by a deliberate project decision and the consequences were tested. With the ensemble, the cold spell scenario predicted 510.9 kW, below the lowest demand in the dataset (606.9 kW), and extreme temperature inputs produced negative demand. A floor of 0 kW at the API level removed the negative values. The thesis concludes that a model that is accurate on a test set must still be tested on scenario inputs before it is used for decisions.

Keywords: Scenario analysis, Decision support, Electricity demand forecasting, Stacking ensemble, Random Forest, Risk assessment

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor, Dr. Nathali Silva ,and my co-supervisor for their valuable guidance, continuous support and constructive feedback throughout this research. Their encouragement helped me to understand the research problem and improve the quality of my work.

I am also thankful to the Department of Information Technology and the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic environment, learning resources and technical support needed to carry out this research. I thank the panel members who reviewed the progress of this project, and whose comments helped me to strengthen the model comparison and the justification of the features.

My heartfelt appreciation goes to my project team members, Thanujan T (IT22545176), Janahan R (IT22069740) and Madushani R.P (IT22273062), for their cooperation, dedication and teamwork. We worked on one shared dataset and one shared system, and their components made my own work more meaningful.

Finally, I would like to thank my family and friends for their continuous encouragement, understanding and moral support throughout the completion of this research

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
ABSTRACT ii
ACKNOWLEDGEMENTS iii
TABLE OF CONTENTS iv
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 3
1.4 Research Gap 3
1.5 Aim and Objectives 4
1.6 Research Questions and Hypotheses 4
1.7 Scope, Limitations and Assumptions 5
1.8 Individual Contribution 5
1.9 Relationship to the Project Proposal 6
1.10 Thesis Structure 7
2 LITERATURE REVIEW AND RELATED WORK 8
2.1 Review Method and Scope 8
2.2 Domain and Theoretical Foundations 8
2.2.1 Electricity demand forecasting 8
2.2.2 Linear models and tree ensembles 8
2.2.3 Stacking ensembles 8
2.2.4 Hyperparameter optimisation 9
2.2.5 Scenario and sensitivity analysis 9
2.2.6 Risk-based decision support 9
2.3 Related Research 9
2.3.1 Load forecasting studies 9
2.3.2 Explainability, anomalies and scenario support 10
2.4 Existing Systems, Products and Techniques 10
2.5 Gap Synthesis and Conceptual Framework 11
3 RESEARCH METHODOLOGY 12
3.1 Research Design 12
3.2 Data and Study Materials 12
3.2.1 Input features 13
3.2.2 Preprocessing, outlier removal and splitting 14
3.3 Experimental Procedure 14

3.3.1 Model comparison 14
3.3.2 Stacking ensemble and out-of-fold predictions 14
3.3.3 Meta learner selection with Optuna 15
3.3.4 Deployment decision rule 15
3.3.5 Scenario analysis engine 16
3.3.6 Risk classification and recommendations 16
3.3.7 Extrapolation and physical validity tests 16
3.4 Measures and Evaluation Metrics 17
3.5 Data Analysis 17
3.6 Ethics, Privacy, Security and Responsible AI 17
3.7 Reproducibility and Research Artefacts 18
4 PRODUCT AND SOLUTION DESIGN 19
4.1 Requirements 19
4.1.1 Stakeholders 19
4.1.2 Functional requirements 19
4.1.3 Non-functional requirements 19
4.2 Component Architecture within the Complete Solution 20
4.3 Design Alternatives and Decisions 20
4.4 Detailed Design 21
4.4.1 Data and model module 21
4.4.2 Stacking module 21
4.4.3 Scenario engine module 21
4.4.4 Risk and recommendation module 21
4.4.5 Service interface 22
4.4.6 Dashboard pages 22
4.5 Standards and Professional Practice 22
5 IMPLEMENTATION, INTEGRATION AND TESTING 23
5.1 Development Environment and Technology Stack 23
5.2 Component Implementation 23
5.2.1 Loading and preparing the data 23
5.2.2 Training and evaluating Random Forest 23
5.2.3 Comparing the three models fairly 24
5.2.4 Measuring feature importance 24
5.2.5 Removing outliers from the training target only 24
5.2.6 Generating out-of-fold predictions 25
5.2.7 Selecting the meta learner with Optuna 25
5.2.8 Building the ensemble and applying the deployment rule 25

5.2.9 Scenario engine and the 0 kW floor 26
5.3 Integration with the Other Components 26
5.4 Verification and Software Testing 27
5.5 Deployment and Operation 27
6 RESULTS AND VALIDATION 29
6.1 Research Results 29
6.1.1 Experimental setting 29
6.1.2 Model comparison and feature importance 29
6.1.3 Stacking ensemble 30
6.1.4 Deployment decision 30
6.1.5 Scenario results 31
6.1.6 Extrapolation and physical validity tests 31
6.2 Product and Technical Results 32
6.3 User or Stakeholder Validation 32
6.4 Integration Results 32
6.5 Summary of Findings 32
7 DISCUSSION 34
7.1 Interpretation of Findings 34
7.2 Comparison with Literature Baselines and Existing Solutions 35
7.3 Research Contribution 35
7.4 Product Contribution 35
7.5 Limitations and Threats to Validity 35
7.6 Implications 36
8 COMMERCIALISATION AND SUSTAINABILITY 37
8.1 Value Proposition and Target Users 37
8.2 Market and Competitive Position 37
8.3 Business or Sustainability Model 37
8.4 Scalability, Intellectual Property and Compliance 38
8.5 Contribution to the Sustainable Development Goals 38
9 CONCLUSION AND FUTURE DIRECTIONS 39
9.1 Conclusion 39
9.2 Achievement of Objectives 39
9.3 Individual Research and Product Contributions 40
9.4 Future Directions 40
REFERENCES 41
GLOSSARY 43
APPENDICES 44

Appendix A Extended Results and Evidence Record 44
A.1 Input features and model settings 44
A.2 Evidence sources in the project repository 44
Appendix B Academic Integrity and AI Use Disclosure 44

1 INTRODUCTION

1.1 Overall Project Context

Electricity cannot be stored cheaply at grid scale, so generation has to follow demand almost instantly. Demand in Sri Lanka changes with the hour of the day, the day of the week, the season, public holidays and weather, and it is also shaped by economic conditions such as income growth and the price of electricity [1], [3]. Poorly anticipated changes lead to excess or insufficient supply, higher operating costs and power outages. Demand forecasting has therefore become an integral part of modern energy management, and artificial intelligence (AI) is now widely used to forecast demand more accurately [1], [3].

The research project **AI-Based Electricity Demand Intelligence System for Sri Lanka** (Project ID R26-IT010) responds to this need. Rather than offering one prediction model on its own, the project aims to provide a decision-support system in which four connected services work on the same data: behaviour analysis of consumption, adaptive demand forecasting, explanation of model predictions, and scenario-based decision support. The system is implemented in Python, and its services are exposed through a FastAPI backend to a web dashboard for energy planners and analysts. All four services use the same underlying dataset of electricity demand, weather and economic information recorded every 15 minutes from January 2020 to May 2025.

The work is carried out by a team of four undergraduates, each responsible for one component. Table 1.1 summarises the components and shows where this thesis fits. The overall architecture adopted by the team is shown in Figure 1.1.

Table 1.1: Components of the complete solution and the component reported in this thesis

Component (service)	Main purpose	Responsibility
Behaviour intelligence service	Clustering of daily consumption patterns and detection of abnormal demand days	Thanujan T (IT22545176)
Adaptive forecasting and prediction service	Short-term demand forecasting and prediction of future load	Janahan R (IT22069740)
Explainable AI (XAI) service	Explanation of demand predictions with SHAP and LIME	Madushani R.P (IT22273062)
Scenario-aware decision support service	Forecasting of demand, what-if scenario simulation, and risk-based recommendations	Dewmini D.M.D.P (IT22630216) (this thesis)

Source: Project proposal, R26-IT010, and topic assessment form.

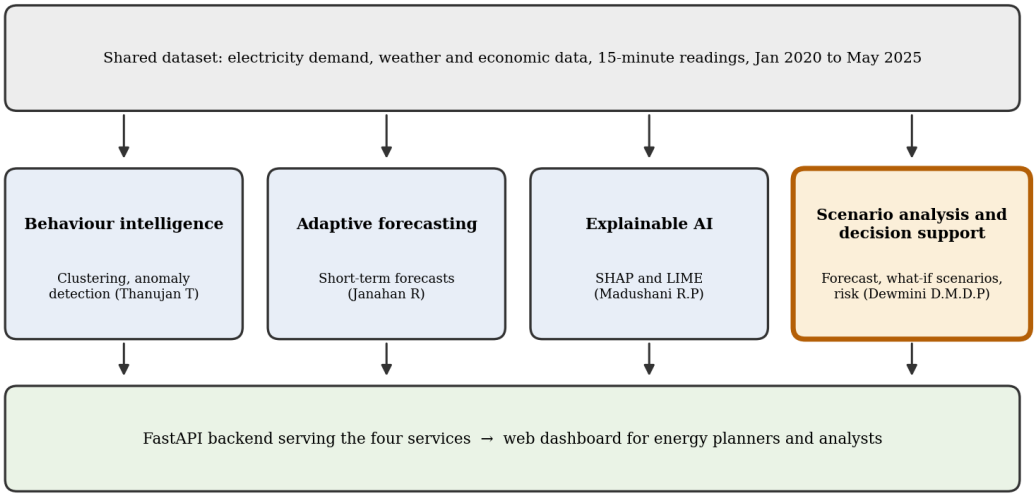


Figure 1.1: High-level architecture of the AI-Based Electricity Demand Intelligence System

Source: Redrawn for this thesis from the project design. The scenario analysis and decision support service, outlined in orange, is the subject of this thesis.

1.2 Individual Component Context

This thesis reports the **scenario analysis and decision support component** of the system. Its task has two connected parts. First, it predicts how much electricity will be used, from weather, economic and time information. Second, it lets a planner test what-if scenarios on top of that prediction, for example *what happens to demand if a heat wave arrives, or if the price of electricity doubles?* The answer is given as a change in demand in kilowatts (kW) and in percent, and it is converted into a risk level and a recommendation.

Four scenarios are built in: a heat wave, a cold spell, a price shock and an economic boom. A custom scenario option accepts any baseline and any modification of the inputs. The forecasting model was developed and compared inside this component, and a stacking ensemble was added as the research extension. The results are served by a FastAPI backend and shown on a React dashboard with four tabs: Overview, Predict, Scenarios and Performance.

The component is related to the other three, but it has a different purpose. The XAI component explains *why* a model made a prediction. The adaptive forecasting component predicts future load with uncertainty. This component changes the inputs of a forecasting model on purpose and measures *how the prediction responds*, so that planners can prepare for conditions that have not happened yet. At the first progress review, the panel noted that the scenario and XAI components appeared to overlap. The difference is that the XAI component studies the model on observed data, while this component studies it on hypothetical data.

1.3 Problem Statement

Energy planners cannot wait for a heat wave or a price shock to happen and then react. They need to estimate the effect of unusual conditions in advance, so that they can prepare supply, adjust pricing or warn the grid. Three weaknesses limit existing forecasting tools in this respect:

- **Single-value predictions.** Most tools report one forecast and cannot compare a baseline with a modified situation, so planners cannot test what-if questions such as how much extra capacity a heat wave would need.
- **Models judged by accuracy alone.** A model with a very high test score can still behave badly when a scenario pushes an input, such as temperature, outside the range seen in training. This is rarely checked.
- **Numbers without decisions.** Outputs are given in kW. They are not turned into risk levels and recommendations that a planner can act on.

The problem addressed by this thesis is therefore: *how can a reliable demand forecasting model be developed and validated so that planners can test what-if scenarios in advance, and how can the scenario results be turned into risk levels and recommendations, while keeping the predictions physically valid when scenario inputs move beyond the range of the training data?*

This problem includes several practical challenges: preparing 15-minute data without damaging real demand patterns; comparing several models fairly before choosing one; deciding whether a more complex model is worth deploying; simulating scenarios and checking that the predictions stay realistic; and delivering the results to planners through an interactive tool.

1.4 Research Gap

Although forecasting research is extensive, the following gaps remain in the context of scenario-based decision support for electricity demand.

- **Forecasting without scenario testing.** Existing tools focus on predicting future demand. Few let planners test what-if situations such as extreme weather, price changes or economic growth before they happen [1], [16].
- **Model selection on accuracy alone.** Models are chosen by test-set error. Their behaviour when scenario inputs fall outside the training range is rarely examined, although this is exactly where decision-support tools are used [13].
- **No link from prediction to decision.** Outputs are numbers in kW and are not translated into risk levels and recommendations that planners can act on.

- **Fragmented systems.** Behaviour analysis, forecasting, explanation and scenario testing are usually built as separate tools instead of one connected system.
- **Limited local tools.** To the best of the author's knowledge, an interactive scenario tool built on Sri Lankan demand data is not widely available.

1.5 Aim and Objectives

Aim. To develop a forecasting and scenario analysis component that helps energy planners make informed, risk-aware decisions about electricity demand, by testing what-if scenarios in advance.

The aim is pursued through the following objectives:

1. To prepare the electricity demand dataset by handling missing values and outliers without letting the test data influence training.
2. To justify the twelve weather, economic and time features used by the model and to measure their influence on demand.
3. To train and compare Linear Regression, Random Forest and Gradient Boosting on identical training and test data.
4. To build a stacking ensemble with out-of-fold predictions and an Optuna-tuned meta learner as a research extension, and to decide with a computed rule whether it should be deployed.
5. To develop a scenario engine that simulates a heat wave, a cold spell, a price shock and an economic boom, and that supports custom scenarios.
6. To classify each scenario result into Low, Medium or High risk and to generate a recommendation.
7. To deliver the component through a FastAPI backend and a React dashboard, and to evaluate it with R^2, RMSE and MAE, including tests of its behaviour on out-of-range inputs.

1.6 Research Questions and Hypotheses

The study is guided by the research questions (RQ) and hypotheses (H) in Table 1.2.

Table 1.2: Research questions and associated hypotheses

ID	Research question	Hypothesis
RQ1	Which of Linear Regression, Random Forest and Gradient Boosting gives the lowest error on the same test set, and does a tuned stacking ensemble improve on them?	H1: All models explain almost all variation in demand, and the ensemble gives only a very small gain over the best single model.
RQ2	How do the four scenarios change predicted demand, and which inputs drive the change?	H2: Temperature-based scenarios (heat wave, cold spell) cause large changes in demand, while price and GDP scenarios cause almost none.
RQ3	Does the deployed model stay physically valid when scenario inputs lie outside the range of the training data?	H3: A model that relies on a linear base learner can predict values outside the observed range, while a tree-based model stays inside it.
RQ4	Can scenario results be turned into risk levels and recommendations and delivered through an API and dashboard?	H4: Each scenario result can be assigned a Low, Medium or High risk level and shown with a recommendation on the dashboard.

Source: Author.

1.7 Scope, Limitations and Assumptions

In scope: preparation of the 15-minute demand dataset; justification of twelve input features; comparison of three regression models; a stacking ensemble with an Optuna-tuned meta learner; a computed deployment rule; four pre-defined scenarios and a custom scenario option; risk classification with recommendations; a FastAPI backend; and a React dashboard.

Out of scope: explanation of why the model predicts a value (the XAI component); behaviour clustering and anomaly detection (the behaviour component); deep-learning and probabilistic forecasting (the forecasting component); causal analysis of demand; a formal user study; and deployment on a live utility system.

Assumptions: the shared dataset is representative of the demand the system would meet in operation; the fixed scenario sizes (for example, price doubled or GDP raised by 20%) are reasonable examples of planning questions; and the results describe how the *model* responds, not how electricity demand is caused in the real world. The main limitations are discussed in Section 7.5.

1.8 Individual Contribution

The individual contributions of the author to the project are summarised in Table 1.3.

Table 1.3: Individual contributions of the author

Area	Contribution
Data preparation	Mean imputation of missing values and training-only IQR outlier removal with a factor of 3.0

Area	Contribution
Model development	Training and comparison of Linear Regression, Random Forest and Gradient Boosting on identical data
Research extension	Stacking ensemble with five-fold out-of-fold predictions and an Optuna study of 50 trials over two meta learner families
Deployment decision	A computed deployment rule, and direct testing of the extrapolation risk after the rule was overridden
Scenario engine	Heat wave, cold spell, price shock and economic boom scenarios, and a live custom scenario option
Decision support	Classification of scenario results into Low, Medium and High risk, with automated recommendations
Delivery	FastAPI backend (model information, prediction, scenarios, scenario test, health check) and a four-tab React dashboard
Safeguard	A floor of 0 kW at the API level so that no prediction can be negative

Source: Author.

1.9 Relationship to the Project Proposal

This thesis follows the research proposal and the first progress review, but the implementation changed in some places as the work progressed. Table 1.4 lists the main differences, so that readers who compare the documents can see what was planned, what was built and why.

Table 1.4: Comparison of the proposal and progress review and the implemented component

Aspect	Proposal / first progress review	This thesis	Reason or status
Forecasting model	Random Forest Regressor (110 trees) with 20 engineered features	Linear Regression, Random Forest and Gradient Boosting compared; stacking ensemble deployed (12 input features)	The panel asked for a model comparison; the ensemble was added as the research extension
Feature justification	Panel noted that the reason for each feature was not clear	Each of the 12 features is justified in Section 3.2.1 and checked with feature importance	Feedback addressed
Outlier handling	IQR, Z-score and Isolation Forest	IQR with a factor of 3.0, applied to the training target only	Wider, gentler rule to keep real high and low demand
Tuning	Optuna, 15 trials, for Random Forest	Optuna, 50 trials, for the stacking meta learner	Tuning effort moved to the research extension
Scenarios	Four pre-defined scenarios and a live custom scenario API	Same four scenarios and custom scenario option	Implemented as planned
Risk and alerts	Low, Medium and High risk with automated recommendations	Implemented; thresholds in Section 3.3.6	Implemented as planned
Backend and frontend	FastAPI and a four-tab React dashboard	Same; a 0 kW floor was added at the API level	Safeguard added after testing

Aspect	Proposal / first progress review	This thesis	Reason or status
Dashboard sliders	Sliders and comparison graphs for scenario outputs	Scenario results are shown as numbers and tables	Not implemented. left as future work
Dataset	Historical CEB demand records, 188,607 rows used	189,888 raw rows; file load_forecasting_dataset_corrected.csv shared by the team (confirmed by notebook 1: 189,888 rows and 15 columns)	188,607 is the number left after the XAI component removed outliers with a different rule (IQR factor 1.5). This component keeps all 189,888 rows; its IQR rule (factor 3.0) removed 0 training rows.
Validation	Accuracy and scenario impact	Model comparison, deployment rule, extrapolation and validity tests; no user study	Time; reported as a limitation

Source: Topic assessment form, first progress review slides (PP1), and this thesis.

1.10 Thesis Structure

Chapter 2 reviews the literature on electricity demand forecasting, ensemble models, scenario analysis and decision support, and synthesises the research gap. Chapter 3 describes the research methodology, data, procedure, evaluation measures and ethical considerations. Chapter 4 presents the requirements and design of the component within the complete solution. Chapter 5 describes implementation, integration and testing. Chapter 6 reports results and validation, and Chapter 7 discusses them, including contributions, limitations and implications. Chapter 8 addresses commercialisation and sustainability. Chapter 9 concludes the thesis and proposes future directions. References, a glossary and appendices follow.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review is a targeted narrative review intended to position the component, not an exhaustive systematic survey. It was organised around the topics *electricity demand forecasting*, *weather and calendar effects on load*, *ensemble and stacking models*, *scenario and sensitivity analysis* and *decision support for energy planning*. Priority was given to review papers and to the original papers that introduced the methods used in this thesis. Numerical results from different studies are not directly comparable because they use different datasets, horizons and spatial levels, and they are therefore not compared numerically in this thesis.

2.2 Domain and Theoretical Foundations

2.2.1 Electricity demand forecasting

Load forecasts are customarily grouped by horizon, from very short-term (minutes to hours) to long-term (years). Statistical models such as ARIMA, exponential smoothing and multiple linear regression can quantify the effect of factors such as temperature and time of day [3], [4], [18]. Machine-learning models such as neural networks, Random Forest and gradient boosting handle non-linear relationships better and are widely used for short- and medium-term forecasting [2], [3]. Reviews of load forecasting consistently identify weather and calendar variables as the main drivers of load, with temperature being the most important weather variable [1], [17]. Reviews also stress that forecasting models must be validated carefully, because a model that fits a dataset well may not generalise [2].

2.2.2 Linear models and tree ensembles

Linear Regression assumes that demand is a straight-line combination of the inputs. It is simple and easy to read, but it has no limit on how far it extrapolates: if an input grows, the prediction keeps growing. A Random Forest averages many decision trees built independently [5]. Gradient Boosting builds trees one after another, each correcting the errors of the earlier ones [6]. A regression tree predicts the average of the training targets in a region of the input space, so the prediction of a tree ensemble stays within the range of target values seen in training [13]. This difference between linear and tree-based models matters for scenario analysis, as shown in Chapter 6.

2.2.3 Stacking ensembles

Stacking combines several base models through a second model called the meta learner [7]. Each base learner makes its own prediction, and the meta learner learns how much to trust each

of them. To avoid data leakage, the meta learner must be trained on out-of-fold predictions, that is, predictions that each base learner made for rows it did not see during its own training [7], [15]. A regularised linear model such as Ridge regression is a common meta learner because it keeps the combination stable [8]. The weights that the meta learner assigns also show which base learners it relies on, which can reveal the structure of the data.

2.2.4 Hyperparameter optimisation

Hyperparameters are settings chosen before training. Optuna is a framework that searches for good settings automatically and records which trial gave the best result [9]. Its default Tree-structured Parzen Estimator (TPE) sampler uses the results of earlier trials to propose more promising settings [10]. The choice of scoring metric matters. When R^2 is close to 1.0 for nearly every candidate, an error measure in the units of the target, such as RMSE in kW, is more sensitive to real differences [14].

2.2.5 Scenario and sensitivity analysis

Scenario analysis asks *what would happen if an input changed?* A baseline input is modified, the model predicts again, and the change is compared with the baseline. It is closely related to sensitivity analysis, which studies how the output of a model responds to changes in its inputs [19]. For a forecasting model, the quality of a scenario answer depends on two things: whether the model learned the real relationship between input and output, and whether the scenario input is still inside the range the model has seen. When it is not, the answer is an extrapolation and should be treated with caution.

2.2.6 Risk-based decision support

A decision-support tool is useful only if its outputs can be acted on. A common practice is to convert a numerical result into a small number of risk levels, each linked to a recommended action, so that a planner does not have to interpret raw numbers every time. This is the approach used in this component. The research design follows the Design Science Research approach, in which an artefact is built and evaluated to solve a practical problem [20].

2.3 Related Research

2.3.1 Load forecasting studies

Hong and Fan [1] give a tutorial review of probabilistic load forecasting and its uses in planning. Ahmad et al. [3] survey load forecasting techniques and their research challenges. Hippert et al. [2] review neural networks for short-term load forecasting and highlight the need for careful validation. Hong et al. [16] describe the Global Energy Forecasting Competition

2014, which set a benchmark for probabilistic energy forecasting, and Hong et al. [17] study how the choice of weather station affects load forecasts. Taylor [18] shows that a double seasonal exponential smoothing model can forecast short-term demand with strong seasonal cycles. Together these studies show that weather and calendar information are central to demand forecasting and that evaluation needs care.

2.3.2 Explainability, anomalies and scenario support

Another group of studies adds explanation to forecasts. Bolstad et al. [23] apply explainable AI to day-ahead load forecasting, using methods such as SHAP [21] and LIME [22]. These studies answer *why* a prediction was made, which is the role of the XAI component in this project. They do not simulate hypothetical conditions. Sensitivity analysis methods [19] are well established in general modelling, but their use as an interactive, risk-aware planning tool for Sri Lankan electricity demand is not widely reported.

Table 2.1: Summary of selected related studies and their relationship to this work

Ref.	Focus of the study	Relationship to this work
[1]	Probabilistic load forecasting, tutorial review	Background on forecasting methods and planning use
[2]	Neural networks for short-term load forecasting	Shows the need for careful validation of forecasting models
[3]	Survey of load forecasting techniques and challenges	Background on statistical, machine-learning and deep-learning models
[16]	Global Energy Forecasting Competition 2014	Benchmark for probabilistic energy forecasting
[17]	Weather station selection for electric load forecasting	Supports the use of weather inputs such as temperature
[18]	Double seasonal exponential smoothing for short-term demand	Statistical baseline; shows daily and weekly cycles
[19]	Global sensitivity analysis	Method basis for changing inputs and measuring the output change
[23]	Explainable AI for day-ahead load forecasting	Explains predictions; this work tests hypothetical inputs instead

Source: Author. Statements are limited to what each study states in its title and abstract.

2.4 Existing Systems, Products and Techniques

The topic assessment form lists commercial and open-source electricity management tools, such as EnergyHub and OpenEnergyMonitor, that mainly provide dashboards of historical trends, alerts for peak loads and simple short-term predictions. Table 2.2 compares the families of approaches that were considered.

Table 2.2: Comparison of approach families for demand forecasting and planning

Approach	Strength	Weakness for scenario-based planning
Monitoring dashboards and alerts	Easy to use; show history and peaks	Do not simulate hypothetical conditions
Statistical models (ARIMA, exponential smoothing)	Simple, well understood [4], [18]	Limited handling of non-linear weather effects; scenario changes are hard to express
Machine-learning forecasting (trees, neural networks)	Capture non-linear effects [2], [3]	Usually return one value; behaviour on out-of-range inputs not checked
Explainable AI add-ons	Explain why a prediction was made [21], [22]	Describe observed behaviour, not hypothetical situations
This component	Forecast, what-if scenarios, risk levels and recommendations in one tool	Limited by the dataset (Section 7.5)

Source: Author, based on the cited studies and the topic assessment form.

2.5 Gap Synthesis and Conceptual Framework

The review shows that demand forecasting is well studied, but that most work and most tools stop at a single prediction. The conceptual framework of this thesis therefore combines (i) a fair comparison of forecasting models, (ii) a stacking ensemble as a research extension, (iii) a computed rule that decides whether the more complex model should be deployed, (iv) a scenario engine with explicit tests of out-of-range behaviour, and (v) risk levels and recommendations delivered through an API and a dashboard. Table 2.3 maps each gap to the objective and design element that responds to it.

Table 2.3: Mapping of research gaps to objectives and design elements

Research gap	Objective	Design element
Forecasting without scenario testing	5	Scenario engine with four scenarios and a custom option
Model selection on accuracy alone	3, 4, 7	Fair model comparison, deployment rule, extrapolation tests
No link from prediction to decision	6	Low, Medium and High risk levels with recommendations
Fragmented systems	7	FastAPI backend and dashboard on the shared dataset
Limited local tools	7	Component built and tested on Sri Lankan demand data

Source: Author.

3 RESEARCH METHODOLOGY

3.1 Research Design

The research follows a Design Science Research (DSR) approach [20]: an artefact (the scenario analysis and decision-support component) is designed, built and evaluated, and the evaluation is carried out as an experimental machine-learning study. The work proceeds in the stages shown in Figure 3.1: data preparation, comparison of candidate models, a stacking ensemble, a deployment decision, scenario simulation, risk classification, and delivery through an API and dashboard. A distinctive feature of the design is that the model is evaluated twice: once on a test set, as usual, and once on scenario inputs, where out-of-range behaviour is tested directly.

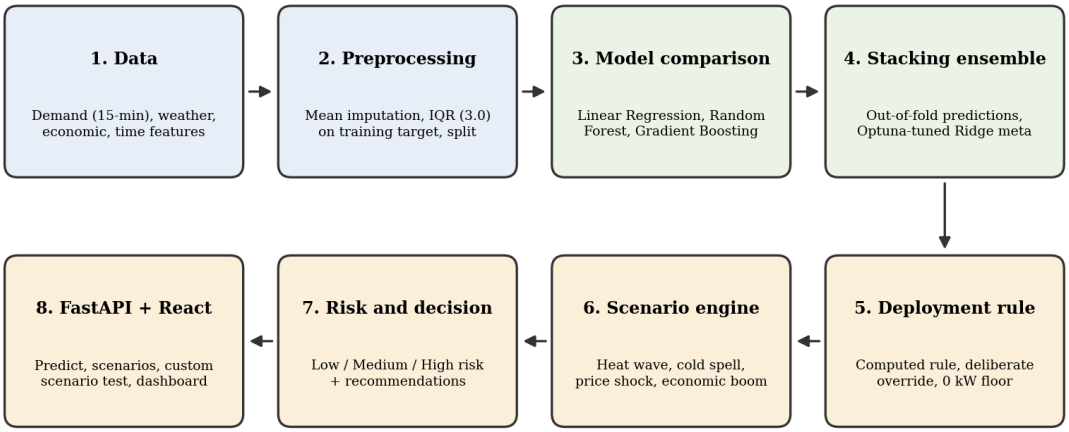


Figure 3.1: Experimental workflow of the component

Source: Author.

3.2 Data and Study Materials

The study uses the file load_forecasting_dataset_corrected.csv, the shared dataset of the project, which is used by all four components. The file contains electricity demand readings taken every 15 minutes from January 2020 to May 2025, giving 189,888 rows. The original public source of this file is described in the XAI component thesis of the project as a Kaggle load-forecasting dataset. In this component, Linear Regression reaches a test R^2 of 1.0, which suggests that the data are synthetic or heavily processed and not raw measured CEB records. The dataset is therefore used as a controlled test bed for the method (Section 7.1). Table 3.1 summarises the dataset and the study configuration.

Table 3.1: Summary of the dataset and study configuration

Item	Description
Records	189,888 observations

Item	Description
Sampling interval	15 minutes
Period	January 2020 to May 2025
Target variable	Electricity demand, Load Demand (kW)
Input features	12 features: weather, economic and time-related variables (Section 3.2.1)
Missing values	Filled with the mean of the column
Outlier handling	IQR rule with a factor of 3.0, applied to the training demand only (0 rows removed, 0.00% of the training set)
Split	80/20 train/test split, random (shuffled, random state 42): 151,910 training rows and 37,978 test rows
Random seed	42 for the models, the cross-validation folds and the Optuna sampler

Source: Author.

3.2.1 Input features

Every row contains the demand value together with twelve features. Each feature was selected because it represents a known driver of electricity use [1], [17], and each was then checked with the feature importance of the Random Forest model (Section 6.1.2). Table 3.2 gives the reason for each feature and the scenario that uses it. This table answers the question raised at the first progress review about why each weather and economic feature was selected.

Table 3.2: Input features, reasons for selection and use in scenarios

Feature	Reason for selection	Used in scenario
Temperature (°C)	Hot weather increases cooling load such as fans and air conditioning; cold weather reduces it. Expected to be the strongest driver.	Heat wave, cold spell
Humidity (%)	Affects how hot the air feels, and so cooling use. It also moves together with temperature in weather events.	Heat wave, cold spell
Wind speed (m/s)	Affects thermal comfort and cooling needs.	Custom
Rainfall (mm)	Influences temperature and daily activity.	Custom
Solar irradiance (W/m²)	Related to the heating of buildings and to solar generation that may reduce net demand.	Custom
GDP (LKR)	Proxy for economic and industrial activity.	Economic boom
Per capita energy use (kWh)	Proxy for the consumption intensity of the population.	Custom
Electricity price (LKR/kWh)	Allows testing whether demand responds to price changes.	Price shock
Day of week, hour of day, month	Capture the daily, weekly and seasonal cycles of demand.	Baseline
Public event	Holidays and events change normal usage patterns.	Custom

Source: Author, based on [1] and [17].

3.2.2 Preprocessing, outlier removal and splitting

Missing values in the features and in the demand column are filled with the mean of the column, so that the models never receive empty cells. The notebook does not print a missing-value count, so this step acts as a safeguard.

Outliers are removed only from the demand column of the training set, using the interquartile range (IQR) rule [12]. If Q1 and Q3 are the 25th and 75th percentiles of training demand and IQR = Q3 − Q1, a row is kept when its demand lies between Q1 − 3.0 × IQR and Q3 + 3.0 × IQR. The factor of 3.0 is wider and gentler than the usual 1.5. Demand legitimately swings a lot, and a stricter bound could remove real high or low demand days instead of genuine errors. The rule is applied to the training data only and never to the test data, so the models are evaluated on the real, untouched problem. The cleaned training set is used for the stacking ensemble.

The data is divided into a training set and a test set. The data is split 80/20 at random (shuffled, random state 42), giving 151,910 training rows and 37,978 test rows. All models use the same split, so the comparison is fair. For time-series data, a split in time order avoids leakage because the model is never trained on readings taken after the readings it is tested on [15].

3.3 Experimental Procedure

3.3.1 Model comparison

Three regression models are trained on the same training data and scored on the same test data: **Linear Regression**, which assumes that demand is a straight-line combination of the inputs; **Random Forest** [5], an average of many independently built decision trees (50 trees, maximum depth 8, random state 42); and **Gradient Boosting** [6], a sequence of trees in which each tree corrects the errors of the previous ones (100 trees, maximum depth 4, random state 42). A single evaluation function computes the same metrics for every model. The Random Forest also records its feature importances, which show how much each feature reduced prediction error.

3.3.2 Stacking ensemble and out-of-fold predictions

As a research extension, the three models become the base learners of a stacking ensemble [7]. Linear Regression is included on purpose. Because it already predicts almost perfectly, the trust that the meta learner places on it becomes a second, independent measure of how linear the data are.

To train the meta learner without leakage, the cleaned training data is split into five folds with shuffling and seed 42. Each base learner is trained on four folds and predicts the fifth, and this is repeated so that every row receives a prediction from a model that never saw it. The three columns of out-of-fold predictions form the training data of the meta learner.

3.3.3 Meta learner selection with Optuna

Two meta learner families were compared inside a single Optuna study [9] of 50 trials, using the TPE sampler [10] with seed 42: Ridge regression [8], a regularised linear model, and Gradient Boosting, a non-linear option that can learn conditional trust between the base learners. Table 3.3 lists the search space. Each trial was scored by 5-fold cross-validated RMSE. RMSE was used instead of R^2 because R^2 is expected to be very close to 1.0 for almost every candidate on this dataset, which leaves little to search over [14].

Table 3.3: Search space of the Optuna study for the meta learner

Hyperparameter	Range	Applies to
meta_type	ridge or gradient_boosting	Both
ridge_alpha	0.001 to 100 (log scale)	Ridge
gb_n_estimators	50 to 300 (step 25)	Gradient Boosting
gb_max_depth	2 to 5	Gradient Boosting
gb_learning_rate	0.01 to 0.30 (log scale)	Gradient Boosting
gb_subsample	0.6 to 1.0	Gradient Boosting

Source: Author's notebook.

The winning meta learner is combined with the three base learners and the same five-fold scheme in a scikit-learn StackingRegressor [11], and trained on the cleaned training data.

3.3.4 Deployment decision rule

To avoid choosing the deployed model by feeling, a computed rule was written. The ensemble replaces Random Forest only if both conditions hold:

- its test R^2 is higher than that of Random Forest by more than 0.0001; and
- the meta learner places less than 10% of its trust on the linear base learner.

The second condition exists because heavy reliance on a linear model brings back the extrapolation risk described in Section 2.2.2, which matters for scenario testing. The rule is a computed condition in the notebook, and its recommendation is printed every time the notebook runs.

3.3.5 Scenario analysis engine

The scenario engine takes a baseline input, changes selected features, asks the deployed model for a new prediction, and reports the change in demand in kW and in percent relative to the baseline. Four pre-defined scenarios are supported (Table 3.4), and a live custom scenario option accepts any baseline and any modification. In the notebook, the baseline is a random sample of 1,000 rows of the dataset (random state 42). Each scenario change is applied to every sampled row, and the reported demand is the mean prediction over the sample. Temperature after the cold spell change is floored at 0 °C, and humidity is kept between 0% and 100%.

Table 3.4: Pre-defined scenarios

Scenario	Change applied to the baseline
Heat wave	Temperature raised and humidity lowered (temperature +8 °C; relative humidity −15 percentage points, floored at 0%)
Cold spell	Temperature lowered and humidity raised (temperature −10 °C, floored at 0 °C; relative humidity +20 percentage points, capped at 100%)
Price shock	Electricity price doubled
Economic boom	GDP raised by 20%

Source: Author's notebook.

3.3.6 Risk classification and recommendations

Each scenario result is classified into a Low, Medium or High risk level from the size of the demand change, and an automated recommendation is attached. This step turns a number in kW into something a planner can act on. Table 3.5 shows the structure of the logic.

Table 3.5: Risk levels and recommendations

Risk level	Condition on the demand change	Recommendation
Low	Change below 5%	Normal operation. keep routine monitoring
Medium	Change from 5% to 20%	Review the reserve capacity and the generation schedule
High	Change above 20%	Prepare extra capacity or contingency plans in advance

Source: Author's implementation.

3.3.7 Extrapolation and physical validity tests

Because scenario inputs often fall outside the training range, two further tests are carried out on the deployed model. First, the prediction of the cold spell scenario is compared with the lowest demand value in the whole dataset and with the prediction of Random Forest. Second, unusual but realistic inputs are typed directly into the prediction form, for example temperatures of 0 °C and −20 °C, to check whether the model can predict negative demand,

which is physically impossible. A floor of 0 kW is then applied at the API level to remove such values.

3.4 Measures and Evaluation Metrics

Predictive performance is measured with three metrics on the test set [14].

- **RMSE** (root mean squared error): the square root of the average squared difference between actual and predicted demand, in kW. It gives more weight to large errors.
- **MAE** (mean absolute error): the average absolute difference between actual and predicted demand, in kW.
- **R^2** (coefficient of determination): the share of the variation in demand that the model explains; 1.0 means a perfect fit.

For the decision part, four further quantities are used: the **change in demand** in kW and in percent between a scenario and its baseline; the **gain in R^2** of the ensemble over Random Forest; the **trust share** that the meta learner places on the linear base learner; and the **risk level** assigned to each scenario.

3.5 Data Analysis

The analysis is carried out at three levels. At the model level, the test metrics of the three models and of the ensemble are compared, and the feature importances of the Random Forest are used to check the feature choices. At the ensemble level, the trust weights of the meta learner and the outcome of the deployment rule are analysed. At the scenario level, the demand change of each scenario is compared with the baseline, the predictions are compared with the observed range of demand, and the risk levels are reviewed. Agreement between the levels, for example a feature that is important in the model and also drives a scenario, increases confidence, and disagreement is reported rather than hidden.

3.6 Ethics, Privacy, Security and Responsible AI

The study uses an existing electricity-demand dataset. It involves no human participants and collects no personal data, so no personally identifiable information is processed. The dataset consists of aggregated, de-identified demand, weather and economic values. The topic assessment form stated that formal ethical clearance is not required for aggregated, anonymised data, and that the work follows responsible data use, transparency in modelling and clear communication of outputs. The Personal Data Protection Act, No. 9 of 2022 [24] is relevant to any commercial use.

The component supports responsible AI in four ways. The deployment rule and its recommendation are printed openly even when overridden. Extrapolated predictions are tested and reported. The 0 kW floor stops physically impossible output. And the results are described as the behaviour of a model, not as proof of cause and effect in the real electricity system. Data leakage is limited by cleaning only the training data and by using out-of-fold predictions for the meta learner. Operational use would need authentication and transport security.

3.7 Reproducibility and Research Artefacts

Reproducibility is supported by fixed random seeds (42), a stated Optuna configuration (50 trials, TPE sampler, seed 42), and the use of the same data for all models. Table 3.6 summarises the reproducibility checklist.

Table 3.6: Reproducibility checklist and status

Item	Status	Note
Dataset described, with size and period	Done	Section 3.2
Dataset source and licence documented	Partly	File name and size documented; original URL and licence still to be added
Train and test split described	Done	Section 3.2.2
Random seeds fixed	Done	Seed 42
Optuna search space and trials stated	Done	Table 3.3, 50 trials
Notebooks run end to end with saved output	Done	Both notebooks hold saved output (notebook 2 executed on 25 August 2026)
Code and notebooks archived	Partly	Notebooks and backend folders exist; repository link not recorded
Library versions recorded	Partly	Python 3.14.6 (notebook metadata); library versions not recorded

Source: Author.

The main research artefacts are the two notebooks (the first for the model comparison and feature importance, and the second for the stacking ensemble and the scenario analysis), the saved model files, the FastAPI backend and the React dashboard. The notebooks are 1_simple_model_training.ipynb and 2_simple_scenario_analysis.ipynb.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Stakeholders

The primary users are energy planners and analysts, who run what-if scenarios to quantify demand impacts before they occur. The secondary users are utility staff and grid managers at organisations such as the Ceylon Electricity Board (CEB), who use pre-computed scenario results and the live scenario endpoint to support load balancing and contingency planning. Policymakers and researchers are tertiary stakeholders, who use the findings to inform energy policy. The other three team members and the project supervisors are also stakeholders, because their components share the data and assess the research.

4.1.2 Functional requirements

Table 4.1: Functional requirements of the component

ID	Requirement	Objective
FR1	Prepare the demand dataset with mean imputation and training-only IQR outlier removal	1
FR2	Justify the twelve features and measure their importance	2
FR3	Train and compare Linear Regression, Random Forest and Gradient Boosting on identical data	3
FR4	Build a stacking ensemble with out-of-fold predictions and an Optuna-tuned meta learner	4
FR5	Compute a deployment recommendation from the R^2 gain and the linear trust share	4
FR6	Simulate the heat wave, cold spell, price shock and economic boom scenarios against a baseline	5
FR7	Accept a custom scenario (any baseline and any modification) and return the change in kW and percent	5
FR8	Classify each scenario result as Low, Medium or High risk and give a recommendation	6
FR9	Never return a negative demand prediction	7
FR10	Provide API endpoints and a dashboard for prediction, scenarios and performance	7

Source: Author.

4.1.3 Non-functional requirements

Table 4.2: Non-functional requirements of the component

Quality	Requirement
Usability	Results are shown in a form that a non-technical planner can read, with changes in kW and percent and a plain recommendation
Reliability	The same inputs and seeds reproduce the same results; predictions are always physically valid (not negative)

Quality	Requirement
Maintainability	Model training, backend and user interface are kept separate so that each can change independently
Performance	Pre-computed scenario results are served quickly; the custom scenario endpoint returns a result for a single request
Transparency	The deployment rule and its recommendation are printed, and extrapolated predictions are identified in the results

Source: Author.

4.2 Component Architecture within the Complete Solution

The component is one service of the larger system in Figure 1.1. It uses the shared dataset, and it does not replace the other components. Figure 4.1 shows its run-time structure. The notebooks prepare the data, compare the models, build the ensemble and run the scenario analysis offline. The trained models are saved to files. The FastAPI backend loads the saved models and exposes endpoints for model information, a single prediction, the four pre-computed scenarios, a custom scenario test and a health check. The React dashboard calls these endpoints. A floor of 0 kW is applied to every prediction at the API level.

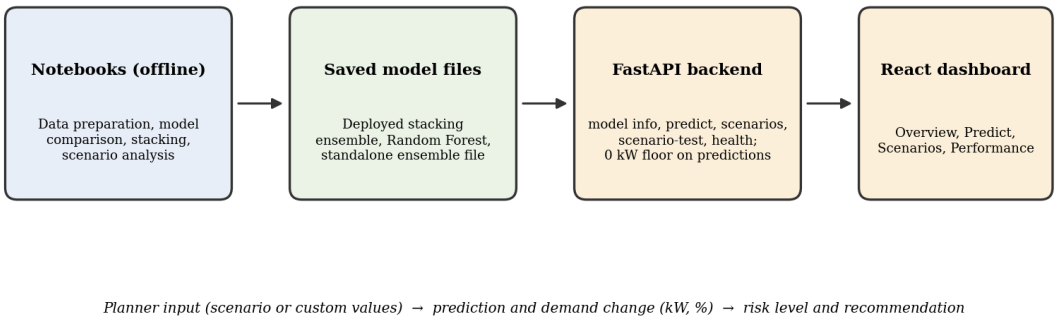


Figure 4.1: Run-time architecture of the component

Source: Author.

4.3 Design Alternatives and Decisions

Table 4.3 records the key design decisions and the reasons for them.

Table 4.3: Key design decisions and justification

Decision	Alternative considered	Justification
Compare three models before choosing	Use Random Forest only	The panel asked for a comparison; it shows how linear the data are
Include Linear Regression as a base learner	Use only tree-based base learners	Its weight in the meta learner is an independent measure of linearity
Out-of-fold predictions for the meta learner	Train the meta learner on in-sample predictions	Avoids leakage from predictions the base learners had already memorised

Decision	Alternative considered	Justification
Score the Optuna study with RMSE	Score with R^2	R^2 is almost 1.0 for every candidate, so RMSE in kW tells the candidates apart better
Wide IQR factor of 3.0 on the training target only	Usual factor of 1.5, or cleaning the test data too	Keeps real high and low demand; keeps the test set untouched
Computed deployment rule	Choose the deployed model by judgement	A computed, repeatable and visible decision
Deploy the stacking ensemble despite the rule	Keep Random Forest deployed, as the rule recommended	Project decision to present the ensemble as the main research contribution; risks tested and mitigated (Section 6.1.6)
Floor of 0 kW at the API level	Retrain a bounded model	Simple and verifiable fix; keeps the ensemble as decided
Keep the ensemble as a separate file	Save only the deployed model	The ensemble stays available to demonstrate even if the deployed model changes
FastAPI and React	A notebook-only output	Planners can use scenarios without running code

Source: Author.

4.4 Detailed Design

4.4.1 Data and model module

This module loads the dataset, fills missing values, removes training outliers with the IQR rule, splits the data and trains the three comparison models with one shared evaluation function. It also records the Random Forest feature importances and writes the metrics used in Chapter 6.

4.4.2 Stacking module

The module creates the five-fold out-of-fold predictions of the three base learners, runs the Optuna study over the two meta learner families, builds the StackingRegressor with the winning meta learner, computes the trust weights, evaluates the deployment rule and saves the ensemble to its own file.

4.4.3 Scenario engine module

The module defines the four scenarios as changes to selected feature values. For each scenario it predicts demand for the baseline and for the modified input, and returns the change in kW and in percent. The same logic serves the custom scenario endpoint.

4.4.4 Risk and recommendation module

The module maps each demand change to a Low, Medium or High risk level using the thresholds in Table 3.5, and attaches an automated recommendation message.

4.4.5 Service interface

The backend is built with FastAPI and provides endpoints for model information, a single prediction, the four pre-computed scenarios, a custom scenario test (/api/scenario-test) and a health check. All endpoints return JSON. Predictions below 0 kW are set to 0 kW before they are returned. The endpoint paths are /api/model-info (model information), /api/predict (single prediction), /api/scenarios (pre-computed scenarios), /api/scenario-test (custom scenario) and /api/health (health check). Features that are missing from a request are set to 0 before the prediction is made, and the last 100 predictions are kept in memory for the model-information endpoint.

4.4.6 Dashboard pages

The React dashboard has four tabs. The **Overview** tab summarises the system and the model The **Predict** tab accepts input values and returns a demand prediction. The **Scenarios** tab shows the four scenarios and lets the user explore custom scenarios, with baseline and scenario results compared. The **Performance** tab shows the evaluation metrics of the model.

4.5 Standards and Professional Practice

The implementation follows common software-engineering and machine-learning practice: modular components, separation of training and test data, fixed random seeds, a documented tuning protocol and a REST/JSON interface between backend and dashboard. The thesis follows the SLIIT dissertation guidelines for structure and formatting and uses IEEE referencing. Responsible-AI practice is followed by printing the deployment recommendation openly, by testing extrapolated predictions, and by stating the limits of what the scenario results can claim.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

Table 5.1 lists the technologies used. Python is the main language for data processing, model training and scenario analysis. scikit-learn provides the models and metrics, Optuna the hyperparameter search, FastAPI the backend and React the dashboard.

Table 5.1: Technology stack and justification

Technology	Role and justification
Python and Jupyter notebooks	Main language; the machine-learning libraries are Python libraries
pandas and NumPy	Loading, cleaning and handling the 15-minute data
scikit-learn [11]	Linear Regression, Random Forest, Gradient Boosting, Ridge, StackingRegressor, cross-validation and the R^2, MAE and RMSE metric functions
Optuna [9]	Automatic search for the meta learner with the TPE sampler
FastAPI	Backend that serves predictions and scenario results as JSON
React	Dashboard with four tabs: Overview, Predict, Scenarios and Performance

Source: Author.

5.2 Component Implementation

This section shows the important code blocks from the notebooks and explains what each one does.

5.2.1 Loading and preparing the data

```
df = pd.read_csv('load_forecasting_dataset_corrected.csv')
features = [
    'Temperature (°C)', 'Humidity (%)', 'Wind Speed (m/s)', 'Rainfall (mm)',
    'Solar Irradiance (W/m²)', 'GDP (LKR)', 'Per Capita Energy Use (kWh)',
    'Electricity Price (LKR/kWh)', 'Day of Week', 'Hour of Day', 'Month',
'Public Event'
]
X = df_clean[features].copy()
y = df_clean['Load Demand (kW)'].copy()
X = X.fillna(X.mean())
y = y.fillna(y.mean())
```

The dataset is loaded into a table. The twelve feature columns are the inputs and the demand column is the target. Missing values in the inputs or in the target are filled with the column mean, so the models never receive empty cells.

5.2.2 Training and evaluating Random Forest

```
rf_model = RandomForestRegressor(n_estimators=50, max_depth=8,
                                 random_state=42, n_jobs=1)
rf_model.fit(X_train, y_train)
y_test_pred = rf_model.predict(X_test)
test_r2 = r2_score(y_test, y_test_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)
```

```
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
```

Random Forest is trained with 50 trees, each no deeper than 8 levels. It predicts the test data, which it has never seen, and the predictions are compared with the real values to calculate R^2, MAE and RMSE.

5.2.3 Comparing the three models fairly

```
def evaluate(name, mdl):
    pred = mdl.predict(X_test)
    model_comparison[name] = {
        'test_r2': float(r2_score(y_test, pred)),
        'test_mae': float(mean_absolute_error(y_test, pred)),
        'test_rmse': float(np.sqrt(mean_squared_error(y_test, pred)))
    }

lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
evaluate('Linear Regression', lin_model)

gb_model = GradientBoostingRegressor(n_estimators=100, max_depth=4,
random_state=42)
gb_model.fit(X_train, y_train)
evaluate('Gradient Boosting', gb_model)
```

One reusable function judges every model in exactly the same way on the same test data. Linear Regression and Gradient Boosting are trained on the same X_train and y_train as Random Forest, which makes the comparison in Table 6.1 fair.

5.2.4 Measuring feature importance

```
feature_importances = dict(zip(available_features,
                               rf_model.feature_importances_.tolist()))
feature_importances = dict(sorted(feature_importances.items(),
                                  key=lambda x: x[1], reverse=True))
```

Random Forest keeps track of how much each feature reduced prediction error while it was being built. Reading this out gives a measured answer to the question of which features matter, instead of a guess.

5.2.5 Removing outliers from the training target only

```
Q1 = y_train.quantile(0.25)
Q3 = y_train.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 3.0 * IQR
upper_bound = Q3 + 3.0 * IQR
outlier_mask = (y_train >= lower_bound) & (y_train <= upper_bound)
X_train_clean = X_train[outlier_mask]
y_train_clean = y_train[outlier_mask]
```

Q1 and Q3 mark the boundaries of the middle 50% of training demand. The wide factor of 3.0 keeps real high and low demand. Rows outside the bounds are left out of the cleaned training set used for the stacking ensemble. The test set is never touched by this step.

5.2.6 Generating out-of-fold predictions

```
cv = KFold(n_splits=5, shuffle=True, random_state=42)
oof_rf = cross_val_predict(base_estimators[0][1], X_train_clean, y_train_clean,
                           cv=cv, n_jobs=1)
oof_gb = cross_val_predict(base_estimators[1][1], X_train_clean, y_train_clean,
                           cv=cv, n_jobs=1)
oof_lr = cross_val_predict(base_estimators[2][1], X_train_clean, y_train_clean,
                           cv=cv, n_jobs=1)
meta_X_train = np.column_stack([oof_rf, oof_gb, oof_lr])
```

cross_val_predict performs the five-fold process described in Section 3.3.2 for each base learner. The three resulting columns become meta_X_train, the input of the meta learner, with no data leakage.

5.2.7 Selecting the meta learner with Optuna

```
def meta_objective(trial):
    meta_type = trial.suggest_categorical('meta_type', ['ridge',
'gradient_boosting'])
    if meta_type == 'ridge':
        alpha = trial.suggest_float('ridge_alpha', 1e-3, 100.0, log=True)
        candidate = make_pipeline(StandardScaler(), Ridge(alpha=alpha,
random_state=42))
    else:
        candidate = GradientBoostingRegressor(
            n_estimators=trial.suggest_int('gb_n_estimators', 50, 300,
step=25),
            max_depth=trial.suggest_int('gb_max_depth', 2, 5),
            learning_rate=trial.suggest_float('gb_learning_rate', 0.01, 0.3,
log=True),
            subsample=trial.suggest_float('gb_subsample', 0.6, 1.0),
            random_state=42
        )
    scores = cross_val_score(candidate, meta_X_train, y_train_clean, cv=5,
                             scoring='neg_root_mean_squared_error', n_jobs=1)
    return -scores.mean()

meta_study = optuna.create_study(direction='minimize',
                                 sampler=optuna.samplers.TPESampler(seed=42))
meta_study.optimize(meta_objective, n_trials=50)
```

Each of the 50 trials picks either a Ridge model with some regularisation strength or a Gradient Boosting model with some settings, trains it on the out-of-fold features, and scores it by 5-fold cross-validated RMSE. Optuna remembers the combination with the lowest error, which is how Ridge was selected.

5.2.8 Building the ensemble and applying the deployment rule

```
stack_tuned = StackingRegressor(
    estimators=base_estimators,
    final_estimator=tuned_meta_learner,
```

```
    cv=cv,
    n_jobs=1
)
stack_tuned.fit(X_train_clean, y_train_clean)

r2_gain = stack_test_r2 - rf_test_r2
deploy_stacking = bool(r2_gain > 0.0001 and linear_weight_share < 0.10)
```

StackingRegressor combines the three base learners, the chosen meta learner and the same five-fold scheme into one model object that is trained and used like any other model. The last line is the deployment rule of Section 3.3.4, a computed condition and not a matter of judgement.

5.2.9 Scenario engine and the 0 kW floor

```
def apply_scenario(df, scenario_type):
    df_mod = df.copy()
    if scenario_type == 'heat_wave':
        df_mod['Temperature (°C)'] += 8
        df_mod['Humidity (%)'] = np.maximum(0, df_mod['Humidity (%)'] - 15)
    elif scenario_type == 'cold_spell':
        df_mod['Temperature (°C)'] = np.maximum(0, df_mod['Temperature (°C)'] -
10)
        df_mod['Humidity (%)'] = np.minimum(100, df_mod['Humidity (%)'] + 20)
    elif scenario_type == 'price_shock':
        df_mod['Electricity Price (LKR/kWh)'] *= 2.0
    elif scenario_type == 'economic_boom':
        df_mod['GDP (LKR)'] *= 1.2
    return df_mod

scenario_pred = model.predict(scenario_features)
change = np.mean(scenario_pred) - baseline_mean
change_pct = (change / baseline_mean) * 100

def _clamp_nonneg(prediction):          # backend/app.py
    return max(float(prediction), 0.0)
```

The scenario function builds a modified copy of the baseline, asks the deployed model for both predictions, and returns the difference. The floor is applied in the API before a prediction is returned.

5.3 Integration with the Other Components

The component shares the raw dataset and the backend conventions with the other three components, as agreed in the integration contract of the project. The same data file is used by all four services, so the results can be compared. In the current build, app.py serves only this component's endpoints and does not call the services of the other three components, so no link to them exists. Linking a scenario date to the XAI explanation is left as future work.

[Insert screenshot: the four tabs of the React dashboard (Figures 5.1a to 5.1d).]

Figure 5.1: Dashboard tabs of the scenario analysis and decision support component

5.4 Verification and Software Testing

Verification covered the data processing, the models, the scenario engine, the API and the dashboard. Table 5.2 lists the checks and the evidence available for each.

Table 5.2: Verification checks and available evidence

Check	What was verified	Evidence
Data preparation	Missing values filled; outliers removed from the training demand only	Notebook output (saved output of notebook 1: 12 features; 151,910 training and 37,978 test rows; IQR bounds 555.64 to 2,443.67 kW; 0 rows removed)
Model evaluation	R^2, MAE and RMSE for the three models on the same test set	Table 6.1
No leakage in the ensemble	Meta learner trained on out-of-fold predictions only	Section 5.2.6
Deployment rule	Rule computed and printed on every run	Table 6.3
Extrapolation	Cold spell prediction compared with the dataset minimum and with Random Forest	Table 6.5
Physical validity	Negative predictions found at 0 °C and −20 °C; 0 kW floor verified on the same inputs	Table 6.6
API	Endpoints return model information, predictions, scenarios, custom scenario results and health	Endpoint code in app.py: /api/model-info, /api/predict, /api/scenarios, /api/scenario-test and /api/health; the 0 kW floor is the function _clamp_nonneg.
Dashboard	Tabs display the returned data correctly with live data	
Notebook re-run	Both notebooks run end to end with saved output	Both notebooks hold saved output (notebook 2 executed on 25 August 2026)

Source: Author.

Results of manual checks are reported here, and no automated test suite is claimed. The notebooks and app.py contain no automated test files.

5.5 Deployment and Operation

In operation, the notebooks are run offline to train the models and to compute the scenario results, and the models are saved to files. The FastAPI backend loads the saved models and serves predictions and scenario results to the React dashboard. The stacking ensemble is the deployed model, and the standalone ensemble file and Random Forest remain available. The system is run locally: the FastAPI backend is served with Uvicorn on port 8000, and the React

dashboard runs on a local development server (port 5173, 5174 or 3000, the origins allowed by the backend).

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Experimental setting

All results come from the 15-minute dataset of 189,888 observations (January 2020 to May 2025). Training-only IQR outlier removal was applied with a factor of 3.0, and all models were trained on the same training set and scored on the same untouched test set. The split was 80/20 at random, giving 151,910 training rows and 37,978 test rows. The stacking ensemble used five-fold out-of-fold predictions and a meta learner chosen by an Optuna study of 50 trials.

6.1.2 Model comparison and feature importance

Table 6.1 compares the three models on the test set. RMSE and MAE measure the typical size of the prediction error in kW (smaller is better), and R^2 measures how much of the variation in demand the model explains (1.0 is a perfect fit).

Table 6.1: Test performance of the three models on the same test set

Model	Test RMSE (kW)	Test MAE (kW)	Test R^2
Linear Regression	almost 0	almost 0	1.0000
Random Forest	0.66	0.21	0.99999
Gradient Boosting	1.09	0.77	0.99997

Source: Author's experiments.

The table must be read carefully. Linear Regression, the simplest model, scored best, and Random Forest scored better than Gradient Boosting. All three models explain almost all of the variation in demand (R^2 of 0.9999 or higher). This tells us something real about this dataset: demand in it is very close to a straight-line function of the inputs, mostly temperature, so there was little complicated, non-linear structure left for a more powerful model to find.

The Random Forest feature importances confirmed that temperature is the dominant feature. In the earlier prototype, temperature-based features accounted for about 88% of the combined importance. In the final notebook the importance of temperature was 1.0000 and that of every other feature rounded to 0.0000 (Table 6.1a). Electricity price and GDP had almost no influence on demand.

Table 6.1a: Random Forest feature importances (final notebook)

Rank	Feature	Random Forest importance
1	Temperature (°C)	1.0000
2	Wind Speed (m/s)	0.0000
3	GDP (LKR)	0.0000

Rank	Feature	Random Forest importance
4	Day of Week	0.0000
5	Month	0.0000
6	Solar Irradiance (W/m²)	0.0000
7	Humidity (%)	0.0000
8	Per Capita Energy Use (kWh)	0.0000
9	Rainfall (mm)	0.0000
10	Electricity Price (LKR/kWh)	0.0000
11	Hour of Day	0.0000
12	Public Event	0.0000

Source: Author's experiments.

6.1.3 Stacking ensemble

Table 6.2: Test performance of the stacking ensemble

Model	Test R^2	Test MAE (kW)	Test RMSE (kW)
Stacking ensemble, default meta learner	1.0000	almost 0	almost 0
Stacking ensemble, Optuna-tuned meta learner	0.99999999999	0.0002	0.0005

Source: Author's experiments.

Ridge regression won the Optuna search against the Gradient Boosting meta learner. The best ridge_alpha was 0.0010 (0.00101) and the best cross-validated RMSE was 0.0006 kW. The tuned meta learner placed about 99.9% of its trust on the Linear Regression base learner, and Random Forest and Gradient Boosting together contributed less than 0.1%. This is a third independent confirmation, after the model comparison and the Random Forest importances, that demand in this dataset is driven almost entirely by a linear relationship with temperature. Even a leak-free, tuned ensemble could not find anything meaningful beyond what the simplest model already had.

6.1.4 Deployment decision

Table 6.3: Outcome of the computed deployment rule

Condition	Required	Measured	Result
R^2 gain of the ensemble over Random Forest	greater than 0.0001	0.000011	Failed
Trust share of the linear base learner	less than 10%	99.9%	Failed
Recommendation of the rule			Keep Random Forest deployed

Source: Author's experiments.

Both conditions failed, so the computed recommendation was to keep Random Forest. The stacking ensemble was nevertheless deployed as the production model by a deliberate project

decision, so that it could be presented as the main research contribution of this component. The notebook still prints the computed recommendation every time it runs, so the override does not hide it. The ensemble is also saved in its own file, independent of whichever model is deployed.

6.1.5 Scenario results

Table 6.4 summarises the scenario results. The earlier prototype used Random Forest with 20 engineered features. The last column uses the deployed stacking ensemble.

Table 6.4: Scenario results

Scenario	Earlier prototype (Random Forest)	Deployed ensemble	Risk level
Heat wave	+19.89% demand	2,310.7 kW (+799.9 kW, +53.0%)	high
Cold spell	−40.33% demand	510.9 kW (−999.9 kW, −66.2%)	high
Price shock (price doubled)	+0.002% demand	1,510.8 kW (about 0 kW, −0.0%)	medium
Economic boom (GDP +20%)	−0.17% demand	1,510.8 kW (about 0 kW, −0.0%)	low

Source: Author's experiments; earlier prototype figures from the first progress review.

The price shock and economic boom scenarios show almost no change in demand. This is not a coding error. It is the model correctly reflecting what it learned, since price and GDP have almost no influence on demand in this dataset. Temperature is what drives the heat wave and cold spell scenarios to show large, real changes. The baseline demand was 1,510.8 kW, the mean prediction over the 1,000-row sample (Section 3.3.5), and the scenario values in Tables 6.4 and 6.5 are mean predictions over the same sample.

6.1.6 Extrapolation and physical validity tests

Because the deployment rule was overridden, the extrapolation risk it warned about was tested directly instead of being assumed away. Table 6.5 shows that it was real.

Table 6.5: Cold spell prediction compared with the observed range

Item	Value (kW)
Lowest demand in the whole dataset (189,888 rows)	606.9
Cold spell prediction, Random Forest deployed	696.9
Cold spell prediction, stacking ensemble deployed	510.9

Source: Author's experiments.

With the ensemble, the cold spell prediction lies below the lowest demand ever recorded, while Random Forest stays safely inside the observed range, because tree splits cannot extrapolate beyond their training bounds. The ensemble's near-linear meta learner can.

Typing unusual but realistic values into the prediction form also showed that the ensemble could predict negative demand (Table 6.6). Demand can never be negative, so a floor was added at the API level: any prediction below 0 kW is set to 0 kW. The fix was verified directly on the same inputs.

Table 6.6: Out-of-range temperature test on the deployed ensemble

Temperature input	Prediction before the floor	Prediction after the floor
0 °C	about −1298 kW	0 kW
−20 °C	about −3297 kW	0 kW

Source: Author's experiments.

6.2 Product and Technical Results

The component delivered a FastAPI backend with endpoints for model information, a single prediction, the four pre-computed scenarios, a custom scenario test and a health check, and a React dashboard with four tabs: Overview, Predict, Scenarios and Performance. Scenario results are classified into Low, Medium and High risk with automated recommendations.

6.3 User or Stakeholder Validation

No formal user study or stakeholder evaluation was carried out for this component. Evidence on whether planners find the scenario results understandable and useful is therefore not available, and any such claim is treated as untested. A usability evaluation with energy planners is proposed as future work.

6.4 Integration Results

The component uses the same dataset and backend conventions as the other three components, so its results can be compared with theirs.

6.5 Summary of Findings

Table 6.7: Summary of findings and strength of evidence

ID	Finding	Strength of evidence
F1	All three models explain almost all variation in demand (R^2 of 0.9999 or higher), and Linear Regression is the most accurate	Strong for this dataset; may not hold for noisier real data (Section 7.1)
F2	The tuned ensemble puts about 99.9% of its trust on the linear base learner, and gains only 0.000011 R^2 over Random Forest	Strong; three independent checks agree
F3	The computed rule recommended keeping Random Forest; the ensemble was deployed by project decision	Documented; the rule is printed on every run
F4	Temperature drives the heat wave and cold spell scenarios; price and GDP scenarios change demand almost not at all	Strong for this dataset; describes model behaviour, not causation

ID	Finding	Strength of evidence
F5	The ensemble predicts a cold spell demand (510.9 kW) below the lowest recorded demand (606.9 kW); Random Forest predicts 696.9 kW	Strong for this scenario; single scenario size
F6	The ensemble can predict negative demand for extreme temperatures; a 0 kW floor removes it	Strong; verified on the same inputs
F7	Scenario results are classified into risk levels and delivered through an API and dashboard	Demonstrated
F8	No user or stakeholder validation has been done	Gap

Source: Author.

7 DISCUSSION

7.1 Interpretation of Findings

Accuracy is not the point of this comparison. The simplest model scored best, which is unusual for electricity demand, where non-linear effects of temperature, time of day and holidays normally let tree ensembles and neural networks outperform linear models [2], [3]. A test R^2 of 1.0 for Linear Regression means that demand in this dataset is almost an exact linear function of the twelve features. This suggests that the dataset may be generated or heavily processed, and its origin should be verified, as the supervisor asked at topic assessment. The comparison is therefore best read as a study of method on a clean, near-linear dataset, and not as a claim about real utility data.

Three checks give the same answer. The plain model comparison, the Random Forest feature importances and the trust weights of the tuned ensemble all show that demand is driven almost entirely by a linear relationship with temperature. The ensemble gained only 0.000011 in R^2 over Random Forest. This is the honest outcome of the research extension: a properly built, leak-free, tuned ensemble found nothing meaningful beyond what the simplest model already had.

Accuracy is not enough for scenario work. All models scored above 0.9999 R^2, yet they behave very differently once scenario inputs leave the training range. The deployed ensemble placed its cold spell prediction below any recorded demand and, for extreme temperatures, below zero. A test-set score could not reveal this risk. Only the direct scenario test did. The 0 kW floor makes the output physically valid, but it does not remove the underlying uncertainty: a prediction outside the range of recorded demand cannot be checked against data and should be read as an extrapolated estimate.

The deployment override was a considered decision. The computed rule recommended Random Forest, and the result tables show why. Deploying the ensemble anyway was a deliberate choice to present it as the research contribution of the component. The consequence was tested instead of assumed, the risk was found to be real, and a fix was applied without hiding the original recommendation. Where the most conservative scenario values are needed, Random Forest can be used, because it stays inside the observed range.

Price and GDP. The near-zero response to a doubled price and a 20% rise in GDP describes this dataset. It should not be read as proof that real Sri Lankan demand is insensitive to price

or economic growth. The finding may partly come from how these variables were generated or from how slowly they change compared with 15-minute readings.

7.2 Comparison with Literature Baselines and Existing Solutions

Earlier studies have focused on forecasting accuracy [1], [3], validation of forecasting models [2], probabilistic forecasting [16] and the role of weather inputs [17]. Others add explanation to forecasts [23]. This work follows the same direction and extends it in three ways. It compares models and then tests the chosen model on scenario inputs, not only on a test set. It uses a computed deployment rule and reports its outcome openly. And it converts scenario results into risk levels and recommendations that are delivered through an API and a dashboard. Numerical results are not compared with those studies because the datasets, horizons and models differ.

7.3 Research Contribution

- A documented comparison of Linear Regression, Random Forest and Gradient Boosting on identical data, with an honest account of why the best-scoring model was not the one expected to be safest for scenarios.
- A leak-free stacking ensemble with a meta learner tuned by Optuna, whose trust weights give an independent measure of how linear the data are.
- A computed deployment rule, and an account of what happened when it was overridden, including the extrapolation and negative-demand findings.
- A scenario analysis with explicit tests of out-of-range behaviour, which shows that test-set accuracy is not enough for decision-support models.

7.4 Product Contribution

The product contribution is a scenario and decision-support tool that planners can open: a FastAPI service and a four-tab React dashboard, four ready-made scenarios, a custom scenario option, and risk levels with recommendations. It turns a forecasting model into a tool for planning questions such as how much extra capacity a heat wave would need.

7.5 Limitations and Threats to Validity

Table 7.1: Limitations and threats to validity

Limitation	Description	Effect on conclusions
Near-linear dataset of unverified origin	Linear Regression reaches R^2 of 1.0; the origin of the data is not documented	The model comparison and the dominance of temperature may not hold for real, noisier data

Limitation	Description	Effect on conclusions
Split of the data	The split was random (80/20), not in time order, so neighbouring 15-minute readings can appear in both the training and test sets. This can make the test scores look better than they would on future data.	Test scores may look better than they would on future data
Heavy reliance on a linear base learner	The deployed ensemble puts about 99.9% of its trust on Linear Regression	Predictions far outside the training range are unreliable, even with the 0 kW floor
Floor does not fix uncertainty	The 0 kW floor removes impossible values but not extrapolation error inside the positive range	Cold spell values below the recorded minimum are estimates only
Fixed scenario sizes	Scenarios such as a doubled price or +20% GDP are chosen examples with no probability attached	Results are illustrations, not forecasts of likelihood
Price and GDP effects	Almost no response in this dataset	Cannot be taken as real price or GDP sensitivity
Mean imputation and training-only cleaning	Simple choices; the test set is not cleaned	Test errors reflect the raw test data, which is intended
Single dataset, single seed	One dataset and seed 42	Results may differ on other data
No user evaluation	Usefulness for planners was not tested	Practical benefit is unproven
Risk thresholds	The risk levels use limits chosen by the author: Low is a change below 5%, Medium is 5% to 20%, and High is above 20%. These limits were not checked with energy planners or with real utility data	Risk labels depend on these choices

Source: Author.

7.6 Implications

For planners, the results suggest that on this data temperature is the first thing to watch when preparing for a demand change, that a heat wave needs extra capacity, and that price policy alone should not be relied on to control demand. For researchers, the study is a reminder that a model with an almost perfect test score can still produce impossible values on scenario inputs, so decision-support models should be tested under scenarios before they are trusted. For the project, the scenario engine could use the explanations of the XAI component and the uncertainty intervals of the forecasting component, so that each scenario result is both explained and given a range.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The proposition is a scenario and decision-support layer on top of demand forecasting. It lets planners ask what-if questions in advance, see the change in demand in kW and percent, and receive a risk level and a recommendation. The primary customer persona is the **energy planner or analyst** at a utility such as the Ceylon Electricity Board (CEB), who needs to estimate extra capacity for extreme weather, check the effect of price policy and prepare contingency plans. Secondary users are utility staff and grid managers. Policymakers and researchers are tertiary users who use the findings to inform planning.

8.2 Market and Competitive Position

The primary market is national energy utilities and grid operators, beginning with Sri Lanka and extending to other developing countries. Secondary markets are energy regulators, large industrial users, renewable-energy developers and smart-city planners. Existing commercial and open-source tools mainly offer dashboards, peak alerts and short-term predictions, and they do not provide interactive scenario testing with risk levels (Section 2.4). The competitive advantage of this component is that scenario testing, risk classification and recommendations are built into one tool and tested on local data.

8.3 Business or Sustainability Model

A suitable model for a tool of this kind is a business-to-government annual software subscription for national utilities, business-to-business licensing for private energy companies and large industrial consumers, and one-off consultancy and integration fees for connecting the tool to existing systems. Pricing could be tiered by the number of users and features, with a reduced-price pilot for local utilities.

Costs fall into development costs, data costs (obtaining, cleaning and updating demand, weather and economic data) and operating costs (server maintenance, model retraining and support). Because the models run in a lightweight backend, operating costs are expected to be modest. Table 8.1 is a template for the budget.

Table 8.1: Budget for the scenario analysis component (LKR)

Item	Quantity / duration	Unit cost	Total
Laptop or workstation for model development	1 unit	250,000	250,000
Cloud computing	6 months	5,000	30,000

Item	Quantity / duration	Unit cost	Total
Web hosting or cloud server	12 months	4,500	54,000
Data storage and backup	12 months	1,500	18,000
Documentation and miscellaneous	1 lot	20,000	20,000
Total			372,000

Source: Author's estimates for this draft; to be replaced with the proposal budget (R26-IT010) or actual expenditure.

8.4 Scalability, Intellectual Property and Compliance

Scalability. The models are light, so the service can scale by adding backend instances. Larger datasets increase only the offline training time. More scenarios can be added as new feature changes without changing the design.

Intellectual property. Scikit-learn, Optuna, FastAPI and React are open-source tools, so the protectable value lies in the integrated pipeline, the scenario and risk logic, and the dashboard, and not in the algorithms. Ownership under SLIIT policy and any data-provider terms should be confirmed.

Compliance. The dataset is aggregated and contains no personal data. A commercial deployment should respect applicable data-protection law, for example Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [24], contractual limits on the use of utility data, and the requirements of the sector regulator. Scenario results should always be shown with their limits, so that they are not mistaken for forecasts of likelihood.

8.5 Contribution to the Sustainable Development Goals

The project aligns with the Sustainable Development Goals [25] as follows.

- **SDG 7 (Affordable and Clean Energy).** Testing scenarios in advance supports reliable, efficient management of the electricity grid and makes it easier to plan reserve capacity.
- **SDG 9 (Industry, Innovation and Infrastructure).** The component applies AI to critical energy infrastructure.
- **SDG 13 (Climate Action).** Better planning can reduce unnecessary over-generation and so lower emissions. The size of this effect has not been measured in this study.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the scenario analysis and decision-support component of the AI-Based Electricity Demand Intelligence System for Sri Lanka. Using a 15-minute dataset of 189,888 observations from January 2020 to May 2025, with mean imputation and training-only IQR outlier removal, three models were compared. Linear Regression was the most accurate, with a test R^2 of 1.0, and Random Forest and Gradient Boosting also scored above 0.9999. A stacking ensemble with out-of-fold predictions and an Optuna-tuned Ridge meta learner was built as a research extension. It placed about 99.9% of its trust on the linear base learner and gained only 0.000011 in R^2 over Random Forest, which showed that demand in this dataset is almost entirely a linear function of temperature.

A computed rule recommended keeping Random Forest. The ensemble was deployed by project decision, and testing its scenario behaviour showed the cost: the cold spell prediction (510.9 kW) was below the lowest recorded demand (606.9 kW), and extreme temperatures gave negative demand. A floor of 0 kW at the API level removed the negative values. The scenario engine showed that temperature drives the heat wave and cold spell results, while price and GDP scenarios changed demand almost not at all. Results are classified into Low, Medium and High risk with recommendations and delivered through a FastAPI backend and a React dashboard.

The main lesson is that a model with an excellent test score must still be tested on scenario inputs before it is used for decisions. The limits of the dataset and the lack of a user study mean that the findings should be read as a method demonstration on a near-linear dataset, and not as proof of real-world demand behaviour.

9.2 Achievement of Objectives

Table 9.1: Achievement of the objectives

No.	Objective (abridged)	Status	Evidence
1	Prepare the dataset without test-data influence	Achieved	Sections 3.2.2 and 5.2
2	Justify the twelve features and measure influence	Achieved	Table 3.2, Section 6.1.2
3	Compare three models on identical data	Achieved	Table 6.1
4	Stacking ensemble and computed deployment rule	Achieved	Tables 6.2 and 6.3

No.	Objective (abridged)	Status	Evidence
5	Scenario engine with four scenarios and custom option	Achieved	Table 6.4
6	Risk classification and recommendations	Achieved	Table 3.5
7	Deliver through API and dashboard; evaluate with R^2, RMSE and MAE	Partly achieved: metrics and extrapolation tests reported; no user evaluation	Sections 6.1 to 6.3

Source: Author.

9.3 Individual Research and Product Contributions

The individual research contributions are the fair model comparison, the leak-free stacking ensemble with a tuned meta learner, the computed deployment rule, and the tests of extrapolation and physical validity. The product contribution is the scenario and decision-support service of FastAPI endpoints and a four-tab dashboard with risk levels and recommendations.

9.4 Future Directions

- **Real utility data.** Test the same approach on measured, noisier data from a utility, where non-linear patterns are more likely to exist.
- **Evaluation in time order.** Use a chronological hold-out and rolling evaluation, which gives a more honest estimate of future performance.
- **Bounded models.** Explore bounded or hybrid models, or limits based on the training range, so that extrapolated predictions stay realistic without relying only on a floor.
- **Custom scenarios on the dashboard.** Let users define their own scenarios directly from the dashboard, instead of only the four built-in ones.
- **Uncertainty and explanation.** Combine scenario results with the uncertainty intervals of the forecasting component and the SHAP and LIME explanations of the XAI component.
- **Cost and capacity assessment.** Add cost and renewable-feasibility assessments to the decision-support output, as outlined in the proposal.
- **User evaluation.** Test the dashboard with planners, using structured tasks and questionnaires, to measure understanding, trust and usefulness.

REFERENCES

[1] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," *International Journal of Forecasting*, vol. 32, no. 3, pp. 914–938, 2016.

[2] H. S. Hippert, C. E. Pedreira and R. C. Souza, "Neural networks for short-term load forecasting: A review and evaluation," *IEEE Transactions on Power Systems*, vol. 16, no. 1, pp. 44–55, 2001.

[3] N. Ahmad, Y. Ghadi, M. Adnan and M. Ali, "Load forecasting techniques for power system: research challenges and survey," *IEEE Access*, vol. 10, 2022.

[4] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne, Australia: OTexts, 2018.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[7] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[8] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[9] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[10] J. Bergstra, R. Bardenet, Y. Bengio and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems 24 (NeurIPS)*, 2011.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[13] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[14] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[15] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, 2012.

[16] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli and R. J. Hyndman, "Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond," *International Journal of Forecasting*, vol. 32, no. 3, pp. 896–913, 2016.

[17] T. Hong, P. Wang and L. White, "Weather station selection for electric load forecasting," *International Journal of Forecasting*, vol. 31, no. 2, pp. 286–295, 2015.

[18] J. W. Taylor, "Short-term electricity demand forecasting using double seasonal exponential smoothing," *Journal of the Operational Research Society*, vol. 54, no. 8, pp. 799–805, 2003.

[19] A. Saltelli et al., *Global Sensitivity Analysis: The Primer*. Chichester, U.K.: Wiley, 2008.

[20] A. R. Hevner, S. T. March, J. Park and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[21] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[22] M. T. Ribeiro, S. Singh and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[23] D. A. Bolstad, U. Cali, M. Kuzlu and U. Halden, "Day-ahead load forecasting using explainable artificial intelligence," in *2022 IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT)*, New Orleans, LA, USA, 2022.

[24] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection Act, No. 9 of 2022*, 2022.

[25] United Nations General Assembly, "Transforming our world: the 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

GLOSSARY

Base learner – One of the models whose predictions are combined by a stacking ensemble.

Baseline – The normal input values against which a scenario result is compared.

Data leakage – Use of information from outside the training data when building a model, which makes results look better than they are.

Extrapolation – Prediction for an input outside the range of the training data.

Floor – A lower limit; here, predictions below 0 kW are set to 0 kW.

Gradient Boosting – A model made of trees built one after another, each correcting the errors of the earlier ones.

Hyperparameter – A model setting chosen before training, such as the number of trees.

IQR (interquartile range) – The distance between the 25th and 75th percentiles; used to define outlier boundaries.

Meta learner – The second-level model of a stacking ensemble that learns how to combine the base learners.

Optuna – A tool that automatically searches for the best hyperparameters.

Out-of-fold prediction – A prediction for a row made by a model that was trained without that row.

R^2, RMSE, MAE – Metrics of model accuracy: share of variation explained, root mean squared error and mean absolute error.

Random Forest – A model that averages many independently built decision trees.

Ridge regression – A linear model with a penalty that keeps its weights small and stable.

Risk level – A Low, Medium or High label assigned to the size of a demand change in a scenario.

Scenario analysis – Changing selected inputs of a model and comparing the new prediction with a baseline.

Stacking ensemble – A model in which a meta learner combines the predictions of several base learners.

TPE (Tree-structured Parzen Estimator) – A search strategy that uses earlier trial results to propose better hyperparameters.

APPENDICES

Appendix A Extended Results and Evidence Record

A.1 Input features and model settings

Table A.1: Input features and model settings

Item	Value
Features (12)	Temperature (°C), Humidity (%), Wind Speed (m/s), Rainfall (mm), Solar Irradiance (W/m²), GDP (LKR), Per Capita Energy Use (kWh), Electricity Price (LKR/kWh), Day of Week, Hour of Day, Month, Public Event
Random Forest	50 trees, maximum depth 8, random state 42
Gradient Boosting	100 trees, maximum depth 4, random state 42
Linear Regression	Default settings
Out-of-fold scheme	5 folds, shuffled, random state 42
Optuna study	50 trials, TPE sampler, seed 42; objective: 5-fold cross-validated RMSE
Winning meta learner	Ridge regression (with standard scaling); best ridge_alpha: 0.0010 (0.00101); best cross-validated RMSE 0.0006 kW
Deployment rule	R^2 gain greater than 0.0001 and linear trust share less than 10%

Source: Author's notebooks.

A.2 Evidence sources in the project repository

Table A.2: Project artefacts that hold the reported evidence

Artefact	Content	Location
1_simple_model_training.ipynb	Data preparation, model comparison, feature importance	Project notebooks folder; writes to ../backend/saved_models/
2_simple_scenario_analysis.ipynb	Stacking ensemble, deployment rule, scenario analysis	Project notebooks folder; writes ../backend/outputs/scenario_results.json
Saved model files	Deployed ensemble, Random Forest, standalone ensemble file	backend/saved_models/ (forecast_model.pkl, stacking_ensemble_model.pkl, model_metadata.json)
Backend	FastAPI service	backend/app.py
Dashboard (React frontend)	React dashboard (Overview, Predict, Scenarios, Performance)	frontend/ (React dashboard, src/ holds the pages)

Source: Author.

Appendix B Academic Integrity and AI Use Disclosure

In line with the module policy, this appendix discloses the use of generative AI. The author remains responsible for the accuracy, originality, citations, data and conclusions of the thesis.

Table B.1: Disclosure of generative AI use

Item	Disclosure
Tool	Claude (Anthropic), a generative AI assistant
Purpose	Structuring and drafting thesis text, tables and diagrams from the author's component summary, viva preparation notes, task list, progress review slides and topic assessment form, following the structure of example theses of other team members
Material provided to the tool	The author's component summary, project documents and two example theses used only for their structure and formatting
Verification performed	Numbers, tables and code excerpts were compared with the saved output of 1_simple_model_training.ipynb and 2_simple_scenario_analysis.ipynb and with the code of app.py.
Sections affected	Chapters 1 to 9, the front matter, the glossary and the appendices were drafted and structured with AI assistance. The experimental code and results come from the author's notebooks.
Content not generated by AI	Experimental data, results and measurements; these come from the author's own experiments

Source: Author.